

2 Lower bound and base cases

2.1 Lower bound of moves for all $k \geq 2$

Firstly, we prove the minimum number of moves required to rearrange the coins. Actually, the proof is independent of k , which is the number of coins that can be slid in each move.

Theorem 1. *For $n \geq 3$, it takes at least n moves to reach the goal configuration.*

Proof. Observe that initially, there is only one pair of adjacent coins with different types of coins, i.e. the pair of alternate coins at positions n and $n + 1$. The first move contributes at most one more pair of alternate coins. Each following move can only create at most two more such pairs. Thus, in n moves, at most $1 + 1 + 2(n - 1) = 2n$ pairs of alternate coins can be created. In the goal configuration, there are $2n - 1$ pairs of alternate coins. Consequently, it takes at least n moves to reach the goal configuration. $\square$

With the lower bound we show the optimal solutions for base cases for $k = 2$. Note that there may be more than one optimal solution for each n .

2.2 Base cases for sliding two coins in each move

Theorem 2. *For $k = 2$, the following sequences are optimal solutions for the corresponding cases.*

1. For $n = 4$: $(2 \rightarrow 9), (5 \rightarrow 2), (8 \rightarrow 5), (1 \rightarrow 8)$.
2. For $n = 5$: $(2 \rightarrow 11), (8 \rightarrow 2), (5 \rightarrow 8), (10 \rightarrow 5), (1 \rightarrow 10)$.
3. For $n = 6$: $(2 \rightarrow 13), (8 \rightarrow 2), (4 \rightarrow 8), (9 \rightarrow 4), (12 \rightarrow 9), (1 \rightarrow 12)$.
4. For $n = 7$: $(2 \rightarrow 15), (11 \rightarrow 2), (5 \rightarrow 11), (10 \rightarrow 5), (7 \rightarrow 10), (14 \rightarrow 7), (1 \rightarrow 14)$.

Proof. It is easy to check the moves in Figures 3, 4, 5 and 6. $\square$

	1	2	3	4	5	6	7	8	9	10
	0	0	0	0	X	X	X	X		
1: (2 -> 9)	0			0	X	X	X	X	0	0
2: (5 -> 2)	0	X	X	0			X	X	0	0
3: (8 -> 5)	0	X	X	0	X	0	X			0
4: (1 -> 8)			X	0	X	0	X	0	X	0

Figure 3: Sliding two coins in each move for $n = 4$.

	1	2	3	4	5	6	7	8	9	10	11	12
	0	0	0	0	0	X	X	X	X	X		
1: (2 -> 11)	0			0	0	X	X	X	X	X	0	0
2: (8 -> 2)	0	X	X	0	0	X	X			X	0	0
3: (5 -> 8)	0	X	X	0			X	0	X	X	0	0
4: (10 -> 5)	0	X	X	0	X	0	X	0	X			0
5: (1 -> 10)			X	0	X	0	X	0	X	0	X	0

Figure 4: Sliding two coins in each move for $n = 5$.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14
	0	0	0	0	0	0	X	X	X	X	X	X		
1: (2 -> 13)	0			0	0	0	X	X	X	X	X	X	0	0
2: (8 -> 2)	0	X	X	0	0	0	X			X	X	X	0	0
3: (4 -> 8)	0	X	X			0	X	0	0	X	X	X	0	0
4: (9 -> 4)	0	X	X	0	X	0	X	0			X	X	0	0
5: (12 -> 9)	0	X	X	0	X	0	X	0	X	0	X			0
6: (1 -> 12)			X	0	X	0	X	0	X	0	X	0	X	0

Figure 5: Sliding two coins in each move for $n = 6$.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
	0	0	0	0	0	0	0	X	X	X	X	X	X	X		
1: (2 -> 15)	0			0	0	0	0	X	X	X	X	X	X	X	0	0
2: (11 -> 2)	0	X	X	0	0	0	0	X	X	X			X	X	0	0
3: (5 -> 11)	0	X	X	0			0	X	X	X	0	0	X	X	0	0
4: (10 -> 5)	0	X	X	0	X	0	0	X	X			0	X	X	0	0
5: (7 -> 10)	0	X	X	0	X	0			X	0	X	0	X	X	0	0
6: (14 -> 7)	0	X	X	0	X	0	X	0	X	0	X	0	X			0
7: (1 -> 14)			X	0	X	0	X	0	X	0	X	0	X	0	X	0

Figure 6: Sliding two coins in each move for $n = 7$.

Note that the occupied position of coins of the goal configuration differs from that of the initial configuration. Except for $n = 3$, the coins are shifted to the right by two positions. This property is useful to derive the recursive algorithm for $k = 2$.

3 Algorithms

3.1 Sliding two coins in each move

In this section, we present a recursive and a non-recursive algorithms to generate the optimal solutions of sliding two coins in each move. The recursive algorithm has time complexity $O(n)$ and space complexity $O(n \log n)$. (The recursive one is easier for the reader to understand its design principle.) The non-recursive version is derived from the recursive one, but it has a better space complexity: $O(\log n)$. Both algorithms are shown in Algorithm 1 and 2, respectively.

Algorithm 1 Recursive Algorithm for $k = 2$

```

1: function SLIDINGCOINRECURSIVE( $Start, N$ )
2:    $\triangleright$   $Start$  is the starting position of coins.
3:    $\triangleright$   $N$  is the number of each type of coins.
4:   if  $N < 3$  then
5:     Return "It is infeasible!"
6:   if  $N = 3, 4, 5, 6$  or  $7$  then
7:     Return the corresponding solution
8:      $(p + Start - 1 \rightarrow q + Start - 1)$ ,
9:     where  $(p \rightarrow q)$  denotes a move as
10:    described in Theorem 2.
11:   Print " $(Start + 1 \rightarrow 2 \cdot N + Start)$ "
12:   Print " $(2 \cdot N + Start - 4 \rightarrow Start + 1)$ "
13:   RECURSIVEK2( $Start + 4, N - 4$ )
14:   Print " $(2 \cdot N + Start - 1 \rightarrow Start + 4)$ "
15:   Print " $(Start \rightarrow 2 \cdot N + Start - 1)$ "

```

Algorithm 2 Iterative Algorithm for $k = 2$

```

1: function SLIDINGCOINITERATIVE( $Start, N$ )
2:    $\triangleright$   $Start$  is the starting position of coins.
3:    $\triangleright$   $N$  is the number of each type of coins.
4:   if  $N < 3$  then
5:     Return "It is infeasible!"
6:    $i \leftarrow Start$ 
7:    $n \leftarrow N$ 
8:   while  $n > 7$  do
9:     Print " $(i + 1 \rightarrow 2 \cdot n + i)$ "
10:    Print " $(2 \cdot n + i - 4 \rightarrow i + 1)$ "
11:     $i \leftarrow i + 4$ 
12:     $n \leftarrow n - 4$ 
13:   if  $N = 3, 4, 5, 6$  or  $7$  then
14:     Return the corresponding solution
15:      $(p + i - 1 \rightarrow q + i - 1)$ ,
16:     where  $(p \rightarrow q)$  denotes a move as
17:     described in Theorem 2.
18:    $n \leftarrow n + 4$ 
19:   while  $n \leq N$  do
20:      $i \leftarrow i - 4$ 
21:     Print " $(2 \cdot n + i - 1 \rightarrow i + 4)$ "
22:     Print " $(i \rightarrow 2 \cdot n + i - 1)$ "
23:      $n \leftarrow n + 4$ 

```

Theorem 3. *Algorithm SLIDINGCOINRECURSIVE(1, n) generates the optimal solutions correctly, for $n \geq 3$. Moreover, the algorithm shifts the coins to the right by two positions, except for $n = 3$.*

Proof. We prove it by induction on n . By Theorem 2, it is clear for the base cases: $n = 4, 5, 6$, and 7.

Suppose the theorem holds for some $n > 7$, we want to prove that it hold as well for $n + 4$. To generalize the proof a little bit, we assume that the coins are arranged initially starting from position i as shown in row (1) of Figure 7. After Steps 11 and 12 of the algorithm, we have the configurations as shown in rows (2) and (3), respectively, of Figure 7. Now observe the sub-configuration in the frame of row (3). It is simply a subproblem of size n and

starting at position $i + 4$. By induction hypothesis, the recursive call in Step 13 of the algorithm returns the configuration as shown in row (4) of Figure 7. Finally, by Steps 14 and 15, we reach the goal configuration. $\square$

Step 13 of the SLIDINGCOINRECURSIVE algorithm is a recursive step. Thus, it will build a stack during the execution and the recursive depth is $O(n)$. So the space complexity is $O(n \log n)$, since each stack frame needs to store 2 numbers of size $O(\log n)$. We can improve the space complexity by removing the recursive step. An iterative algorithm is shown in Algorithm 2. It uses two variables, i.e. N and i , to generate the moves. Therefore, the space complexity can be reduced to $O(\log n)$.

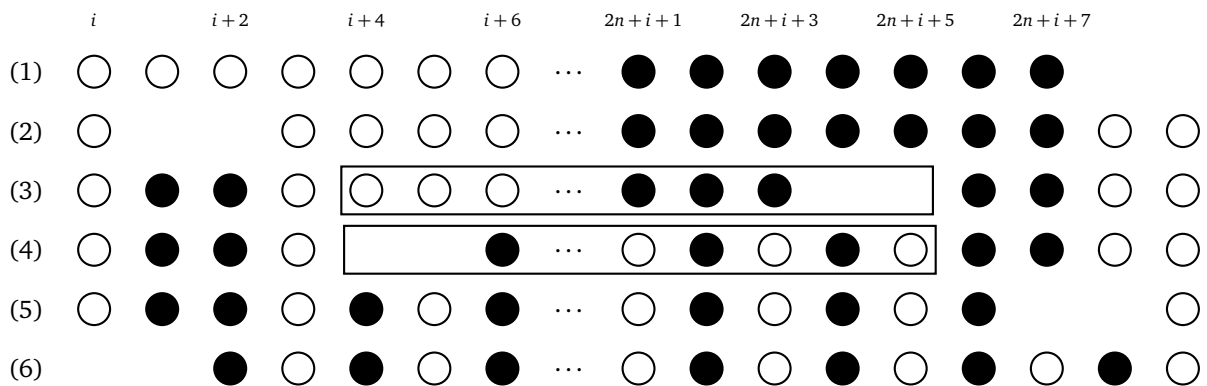


Figure 7: The inductive step for the proof of Theorem 3. Note that the coins are arranged initially at position i and the problem size is $n + 4$.

4 Conclusion

In this paper, we have proved that it takes at least n moves to solve the linear sliding-coins puzzle. Also, we have designed two algorithms to solve the puzzle, which generate optimal solutions under $k = 2$. We further studied the case for $k = 3$. It is interesting that the cases for $k = 2$ and $k = 3$ seem similar while the patterns of their solutions are quite different. How about the case for $k \geq 4$? Could the puzzle be solved in n moves? There are still many other variants of the sliding-coin puzzle. It would be interesting to know to which complexity classes they exactly belong.

References

- [1] M. Abellanas, S. Bereg, F. Hurtado, A. G. Olaverri, D. Rappaport, and J. Tejel, Moving

Coins, Computational Geometry: Theory and Applications, Vol 34 (1), pages 35–48, 2006.

- [2] C.- C. Chu, M.-Y. Lee and C.-S. Tsai, Sliding-Coin Puzzle, unpublished manuscript (in Chinese), 2003.
- [3] E. Demaine, M. Demaine and H. Verrill, Coin-Moving Puzzles, in More Games of No Chance, edited by R.J. Nowakowski, pages 405–431, Cambridge University Press, 2002.
- [4] E. Demaine and M. Demaine, Puzzles, Art, and Magic with Algorithms, Theory Comput. Systems 39, pages 473-481, 2006.
- [5] A. Gardner, Mathematical Puzzling. Dover, Mineola NY, 1999.
- [6] A. Levitin, Introduction to The Design and Analysis of Algorithms, 2nd Edition, Addison Wesley, 2007.